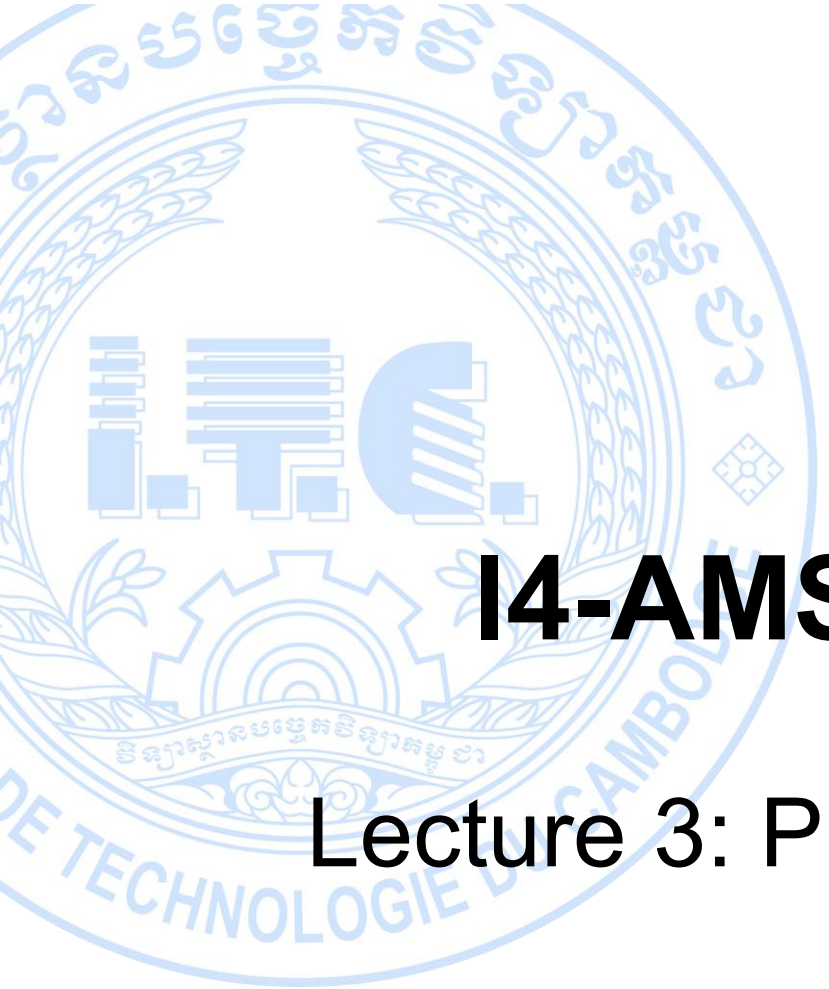


I4-AMS: Artificial Intelligence

Lecture 3: Practical Aspects of Deep Learning

Today’s agenda

Part 1 Setting up your ML application

Train/dev/test sets, bias and variance, the basic recipe, network size

Part 2 Regularization

L2, dropout, data augmentation, early stopping, max-norm

Part 3 Setting up the optimization problem

Input normalization, initialization, vanishing gradients, gradient checking

Part 1

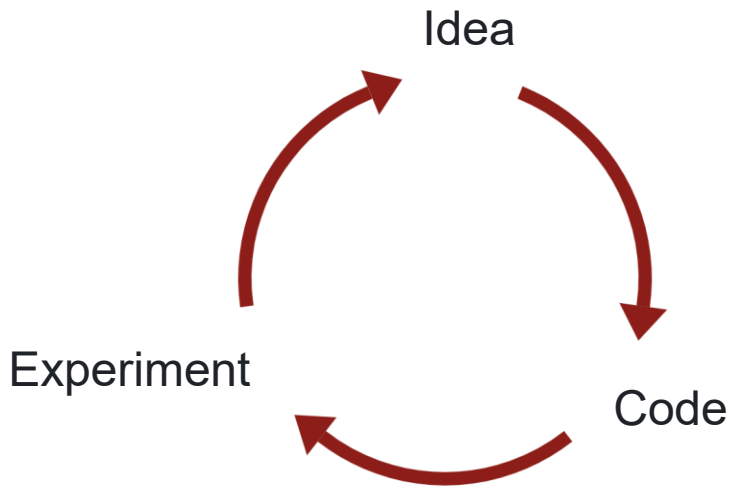
Setting up your ML application

Data splits, bias and variance, the basic recipe, and how big the network should be

Applied deep learning is iterative

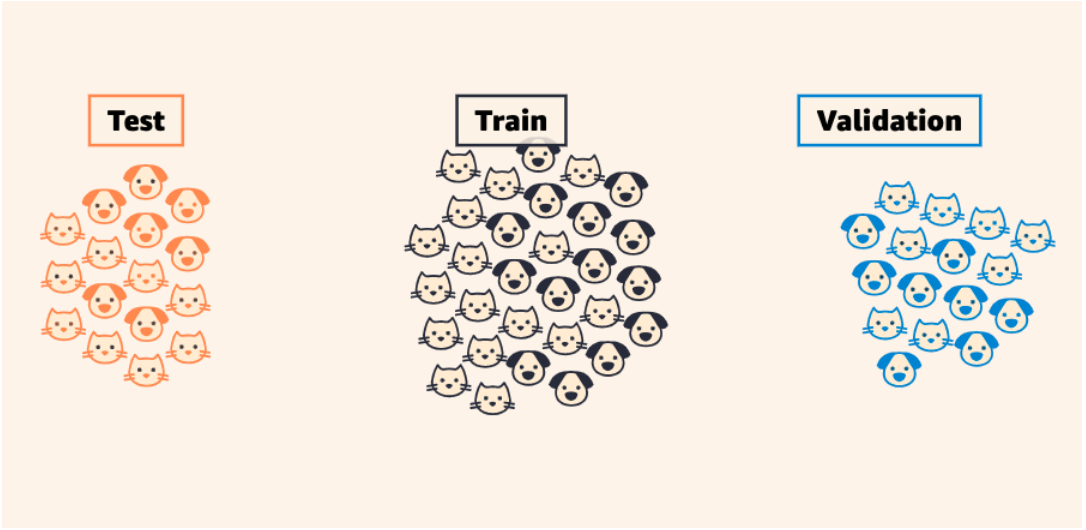
Before training, you must choose:

- the number of layers L
- the hidden units $n^{[l]}$ per layer
- the learning rate α
- the activation functions
- regularization, mini-batch size, optimizer, …



❑ Good values are almost impossible to guess, even for experts, and intuitions rarely transfer between domains (vision, speech, NLP, structured data). What matters is how **fast** you can go around the loop: good data splits and quick diagnostics make every loop shorter.

Train / dev / test sets



Training set

Fit the parameters $\mathbf{W}, \mathbf{b}$ with gradient descent.

Dev set

Also called hold-out or validation set. Compare models and choose hyperparameters.

Test set

Unbiased estimate of the final model. Use it once, at the end.

Small data (100 – 10,000)	train 60%	dev 20%	test 20%
Big data (1,000,000)	train 98%	dev 1%	test 1%

With a million examples, 10,000 dev examples (1%) are enough to tell two models apart: the old 60/20/20 rule wastes data. Even 99.5 / 0.4 / 0.1 can work.

Mismatched train and test distributions

Training set

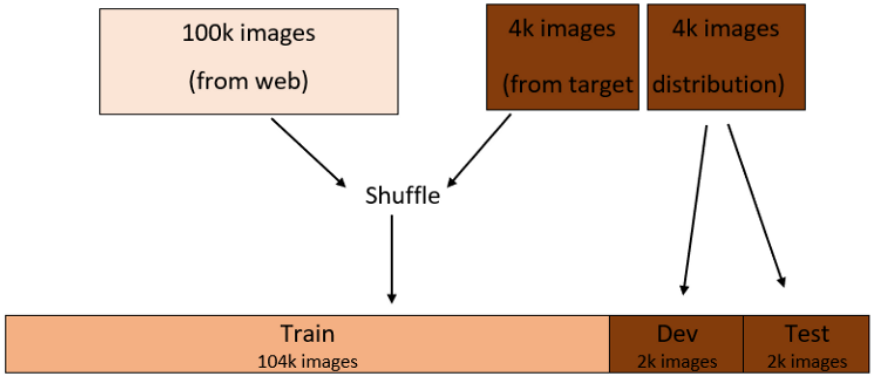
Cat pictures crawled from webpages: high resolution, well framed, professional.

Dev and test sets

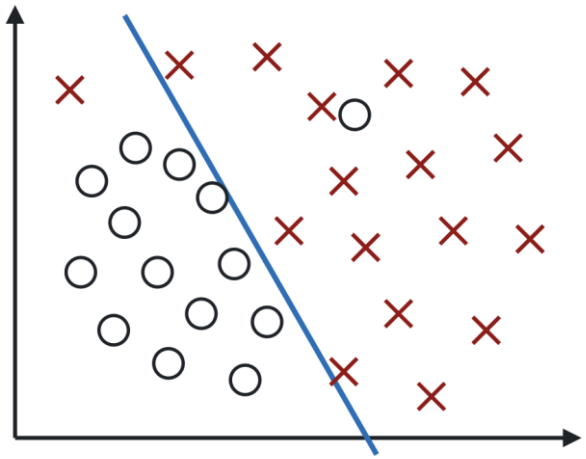
Cat pictures uploaded by the users of your app: blurry, phone camera, poor light.

> **Rule:** make sure the dev and test sets come from the **same** distribution: the data you expect in the real application. The training set may differ (more data from other sources often helps).

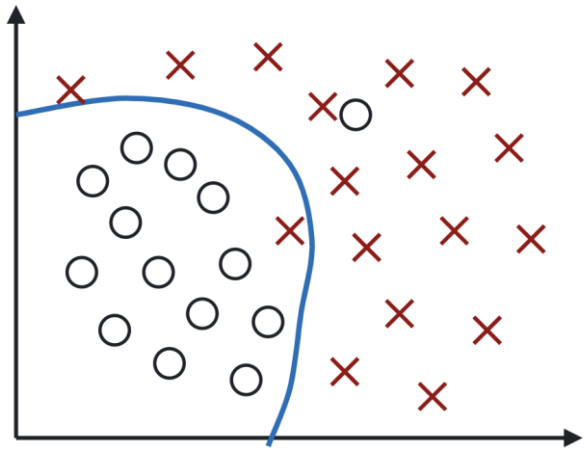
- You aim at the dev set when you tune; the test set must measure the same target.
- Not having a test set can be acceptable if you do not need an unbiased estimate. People then often call the dev set "test set": be careful with the names.



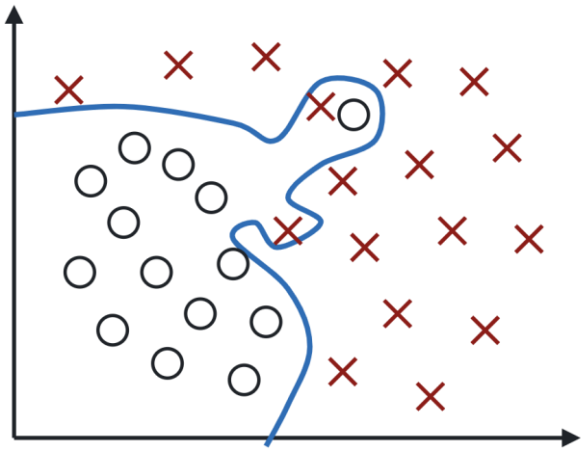
Bias and variance



High bias
underfitting



Just right

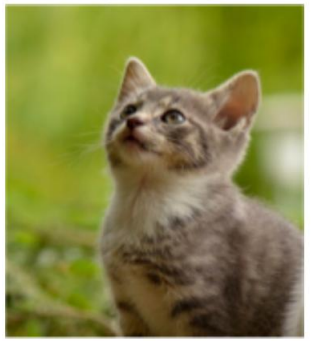


High variance
overfitting

With two features we can look at the decision boundary. With thousands of features, we cannot, so we look at two numbers instead: the **training error** and the **dev error**.

Diagnosing bias and variance

Cat classification. Assume humans make almost no errors: optimal (Bayes) error ≈ 0%.





	Case 1	Case 2	Case 3	Case 4
Train set error	1%	15%	15%	0.5%
Dev set error	11%	16%	30%	1%
Diagnosis	high variance	high bias	high bias and high variance	low bias, low variance

Bias

Compare the **training** error with the optimal error.

Variance

Compare the **dev** error with the training error.

> If the images were so blurry that even the best possible classifier had 15% error, case 2 would be a good model. The diagnosis is always relative to the optimal error.

A basic recipe for machine learning

High bias?
Training set performance

yes →

- Bigger network
- train longer
- (try another architecture)

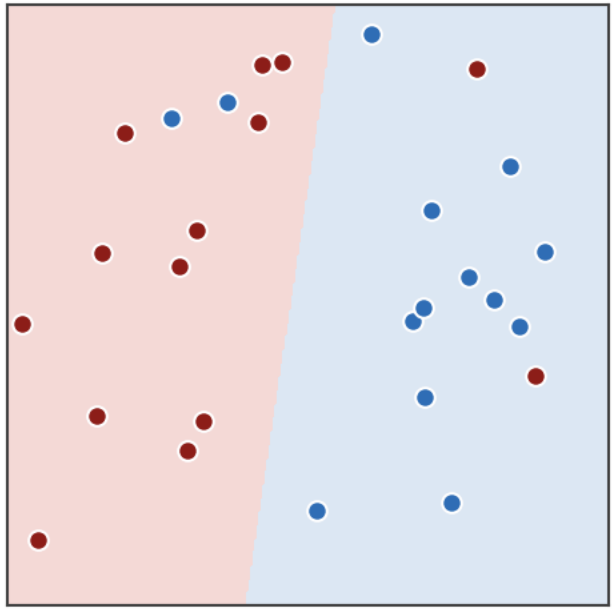
High variance?
Dev set performance

yes →

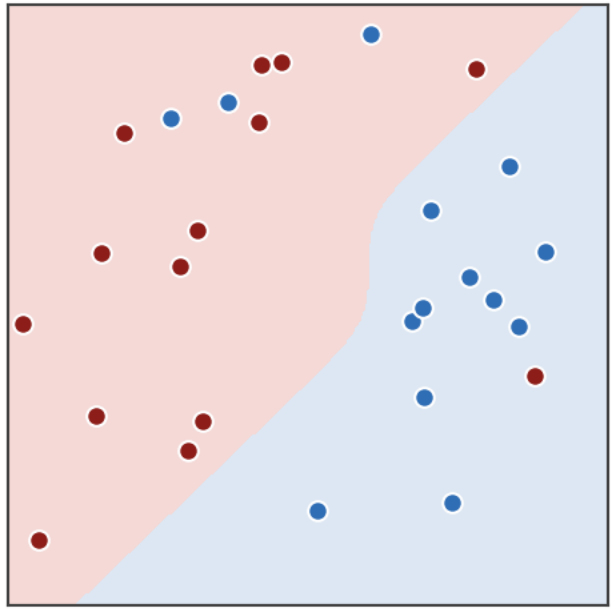
- More data
- Regularization
- (try another architecture)

After each fix, go back to the top. In deep learning the bias–variance "tradeoff" is weaker: a bigger network reduces bias without hurting variance (if regularized), and more data reduces variance without hurting bias.

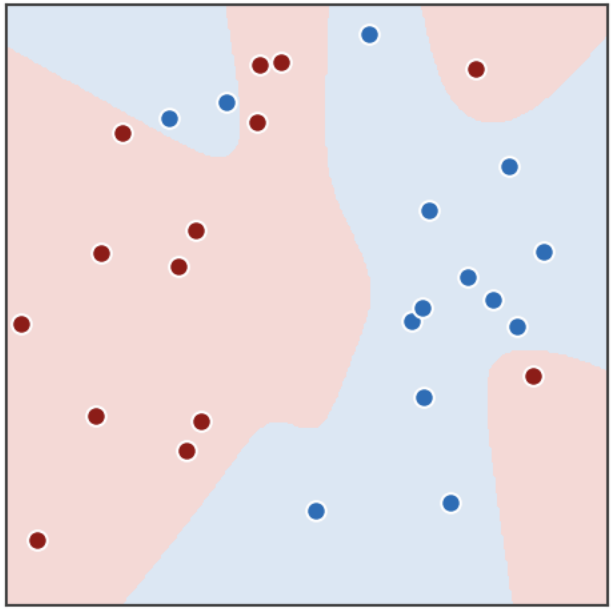
Network size and capacity



1 hidden unit
a straight boundary



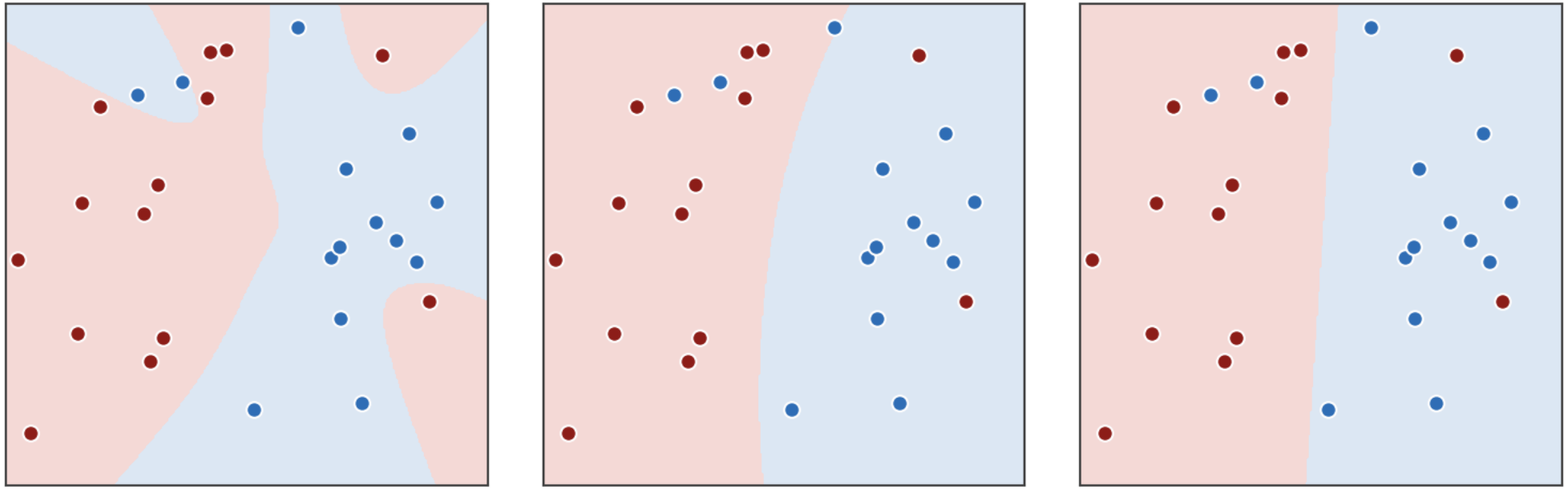
2 hidden units
a bent boundary



20 hidden units
fits every point, even the noise

More hidden units = more **capacity**: the network can represent more complicated functions. A blessing (complex data) and a curse (it can also fit the noise: overfitting).

Bigger network, stronger regularization



λ = 0.001 **λ = 0.01** **λ = 0.1**

The same 20-unit network: increasing the L2 strength λ (Part 2) gives smoother and smoother boundaries.

> Do not use a smaller network because you are afraid of overfitting. Use as big a network as your compute allows, and control overfitting with **regularization**.

Part 2

Regularization

L2 regularization, dropout, data augmentation, early stopping, and max-norm



L2 regularization: logistic regression

$$\min_{\mathbf{w},\, b} J(\mathbf{w}, b), \qquad J(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}\left(\hat{y}^{(i)}, y^{(i)}\right) + \underbrace{\frac{\lambda}{2m} \|\mathbf{w}\|_2^2}_{\text{new}}$$

L2 regularization (the usual choice)

$$\|\mathbf{w}\|_2^2 = \sum_{j=1}^{n_x} w_j^2 = \mathbf{w}^\top \mathbf{w}$$

L1 regularization

$$\frac{\lambda}{2m} \sum_{j=1}^{n_x} |w_j| = \frac{\lambda}{2m} \|\mathbf{w}\|_1$$

Makes $\mathbf{w}$ sparse (many zeros); rarely used for neural networks.

- λ is the **regularization hyperparameter**, tuned on the dev set.
- We do not regularize b: it is one number, while $\mathbf{w}$ has n_x; adding $\frac{\lambda}{2m} b^2$ changes almost nothing.

L2 regularization: neural network

$$J\left(\mathbf{W}^{[1]}, \mathbf{b}^{[1]}, \ldots, \mathbf{W}^{[L]}, \mathbf{b}^{[L]}\right) = \frac{1}{m}\sum_{i=1}^{m}\mathcal{L}\left(\hat{y}^{(i)}, y^{(i)}\right) + \frac{\lambda}{2m}\sum_{l=1}^{L}\|\mathbf{W}^{[l]}\|_F^2$$

Frobenius norm, the sum of the squares of all the entries: $\|\mathbf{W}^{[l]}\|_F^2 = \sum_{i=1}^{n^{[l-1]}}\sum_{j=1}^{n^{[l]}}\left(W_{ij}^{[l]}\right)^2$

Gradient

$$d\mathbf{W}^{[l]} = (\text{from backprop}) + \frac{\lambda}{m}\mathbf{W}^{[l]}$$

Update: "weight decay"

$$\mathbf{W}^{[l]} := \mathbf{W}^{[l]} - \alpha\, d\mathbf{W}^{[l]}$$
$$= \left(1 - \frac{\alpha\lambda}{m}\right)\mathbf{W}^{[l]} - \alpha\,(\text{from backprop})$$

Each step first shrinks $\mathbf{W}^{[l]}$ by the factor $1 - \frac{\alpha\lambda}{m} < 1$. In PyTorch: torch.optim.SGD(params, lr, weight_decay=λ/m).

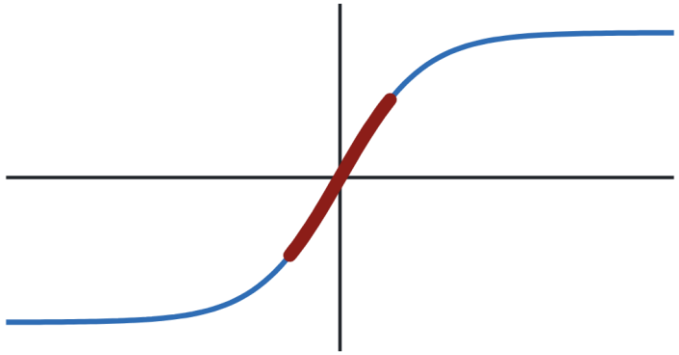
Why does regularization reduce overfitting?

A simpler network

A large λ pushes $\mathbf{W}^{[l]} \approx 0$: many hidden units have almost no effect, and the network behaves like a much smaller one.

λ too large: high bias. $\lambda = 0$: high variance. An intermediate λ is "just right".

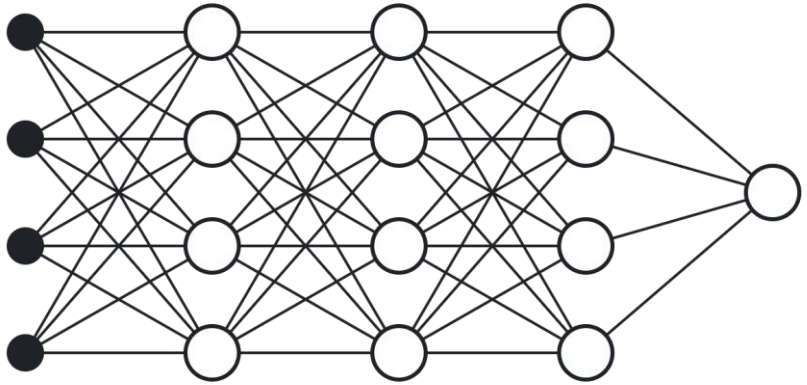
An almost linear network



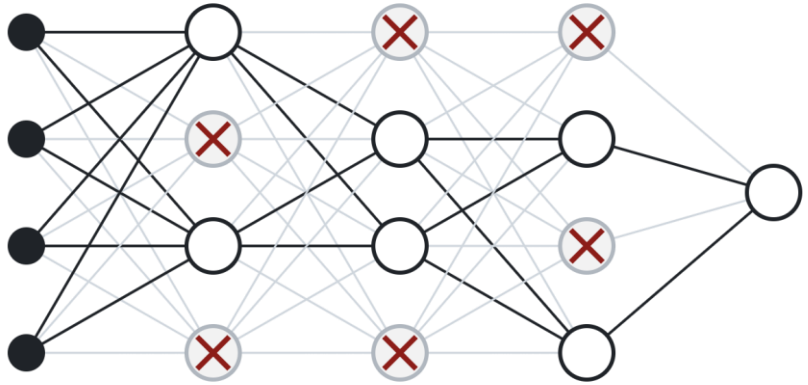
Small $\mathbf{W} \to$ small $\mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]}\mathbf{W}^{[l]} + \mathbf{b}^{[l]} \to \tanh$ works in its linear region (red): each layer is almost linear, so the network computes a simpler function.

Debugging tip: plot J **including** the regularization term. The data term alone may not decrease at every iteration.

Dropout regularization



Full network



One training example, keep_prob = 0.5

- For each training example, keep each hidden unit with probability keep_prob, and remove the others with their connections.
- Each example trains a different, much smaller "thinned" network.
- Careful with PyTorch: nn.Dropout(p) takes the probability of **dropping**, p = 1 − keep_prob.

Why does dropout work?

> **Intuition:** a unit cannot rely on any one input, because any input can disappear. So, it spreads out its weights over all its inputs, which shrinks them, like L2.

Choosing keep_prob

- A different keep_prob per layer: lower (e.g. 0.5) for large layers that overfit most
- 1.0 for small layers (no dropout)
- For the input layer: 1.0 or close (0.9)

Downsides

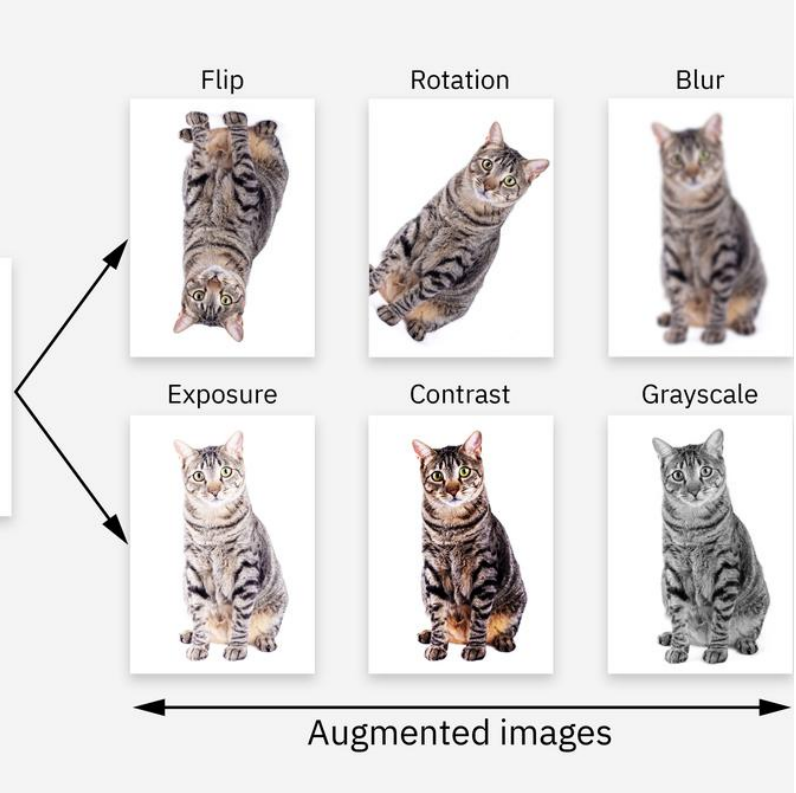
- J is no longer well defined: the network changes at every step
- So, first check that J decreases with keep_prob = 1, then turn dropout on
- More hyperparameters (one per layer)

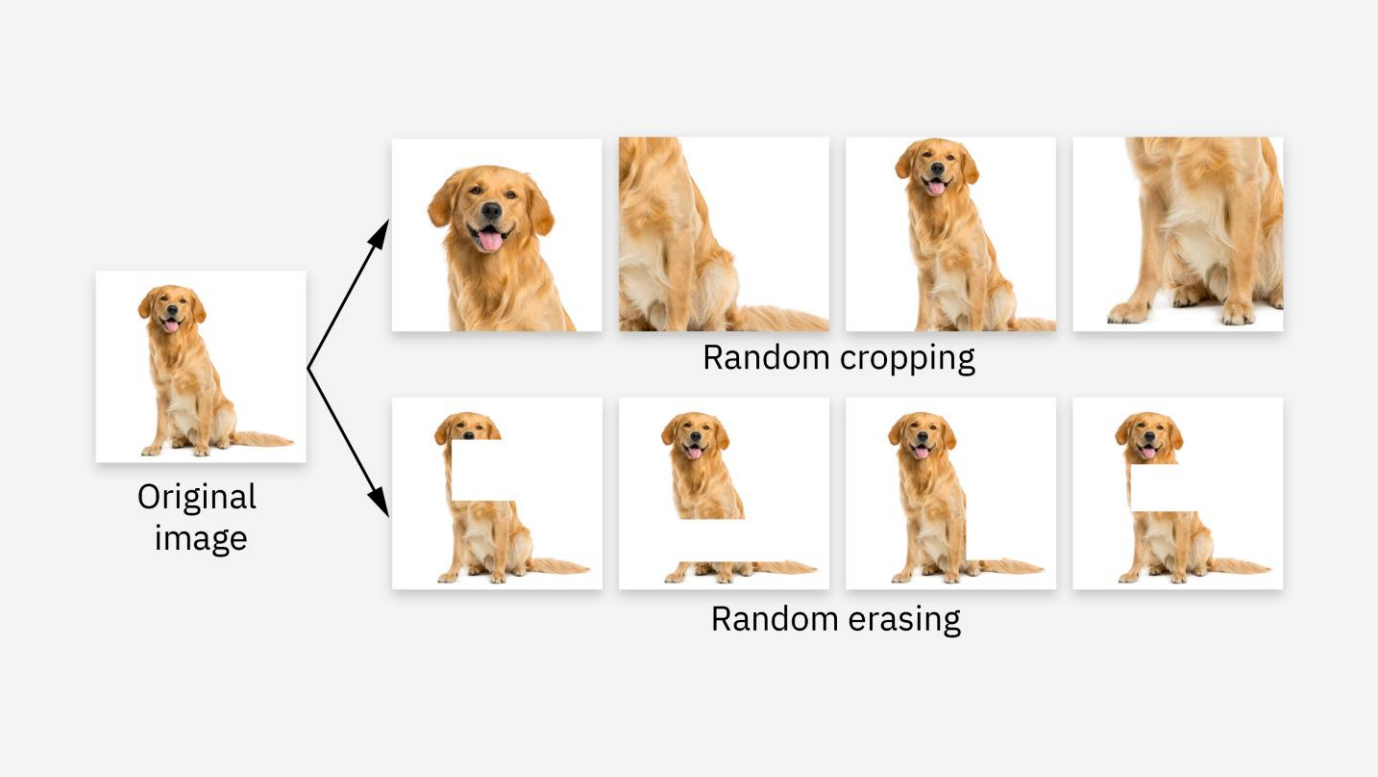
Use dropout when you overfit. It is very common in computer vision, where there is almost never enough data.

Other regularization methods

❑ Data augmentation

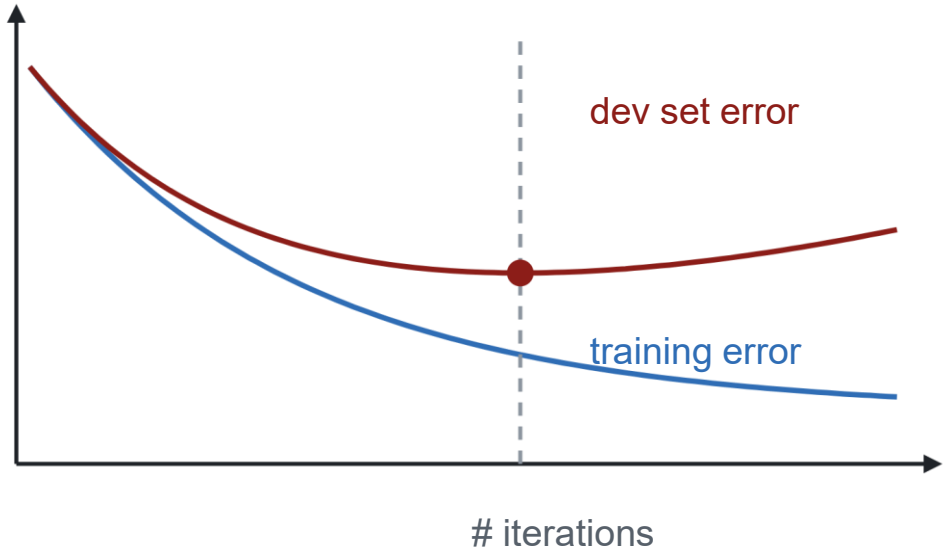
- Create new examples from existing ones: flips, random crops, small rotations, distortions
- Cheap extra data, less overfitting





Other regularization methods

❑ Early stopping



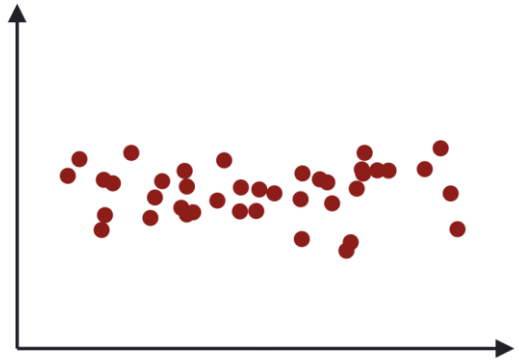
- **Prevents Overfitting:** Stops the model before it memorizes noise and specific patterns in the training set.
- **Saves Resources:** Cuts down unnecessary training time and computational power once performance plateaus.

Part 3

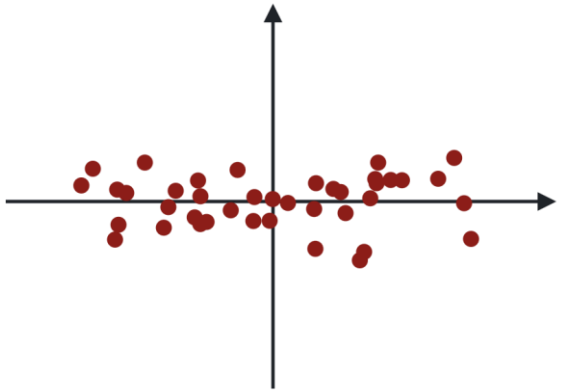
Setting up the optimization problem

Input normalization, weight initialization, vanishing and exploding gradients, gradient checking

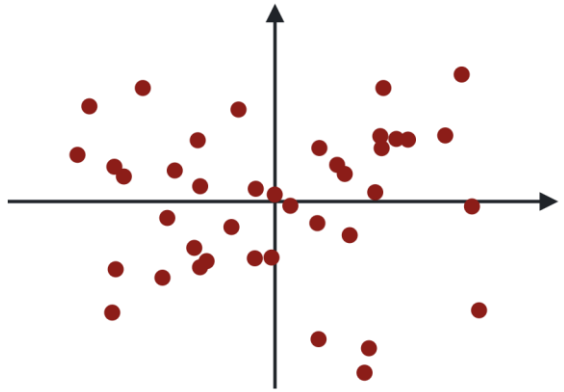
Normalizing inputs



Raw data



Subtract the mean



Divide by the standard deviation

$$\boldsymbol{\mu} = \frac{1}{m}\sum_{i=1}^{m}\mathbf{x}^{(i)}$$

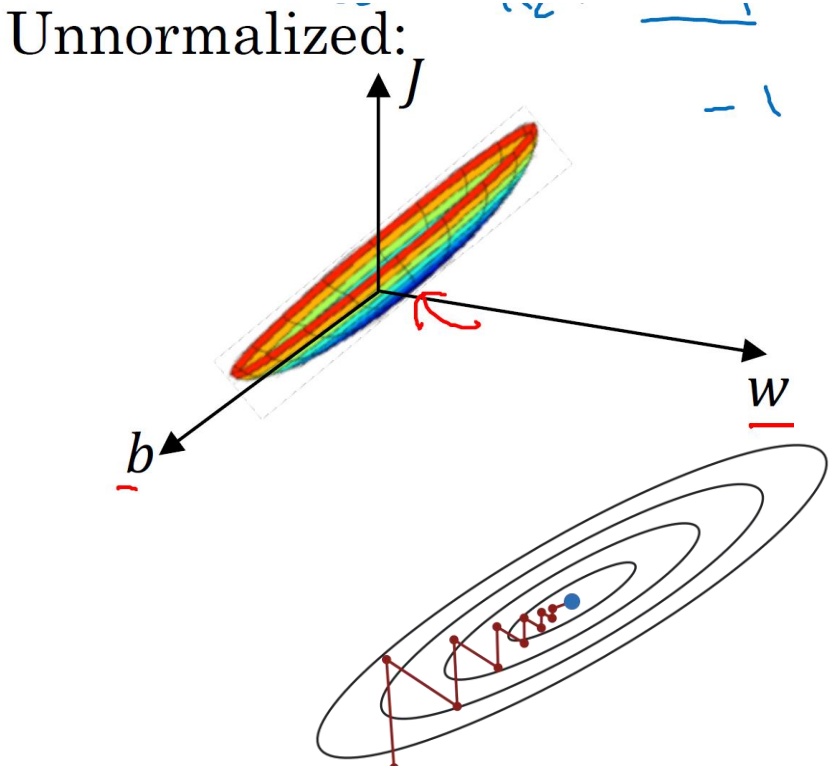
$$\boldsymbol{\sigma}^2 = \frac{1}{m}\sum_{i=1}^{m}\left(\mathbf{x}^{(i)} - \boldsymbol{\mu}\right)^2$$

$$\mathbf{X} := \frac{\mathbf{X} - \boldsymbol{\mu}}{\boldsymbol{\sigma}}$$

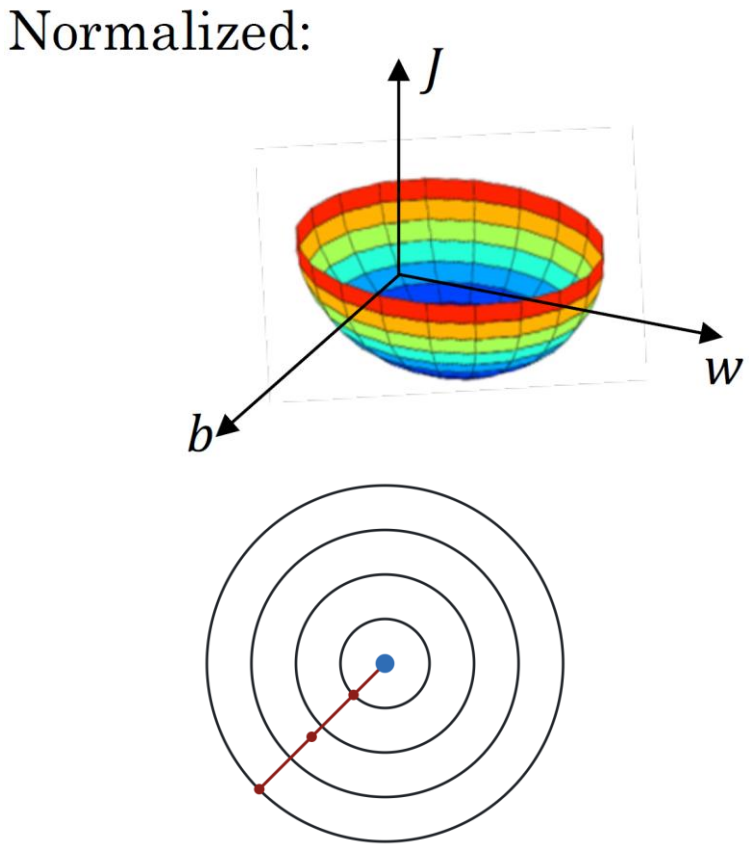
$\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ have shape $(1, n_x)$, one value per feature; the operations are elementwise and broadcast over the rows of $\mathbf{X}$.

> Use the **same** $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$, computed on the training set, to normalize the dev and test sets.

Why normalize inputs?



Unnormalized: $x_1 \in [1, 1000]$, $x_2 \in [0, 1]$. The cost is a long, narrow valley: only a small α works, and gradient descent zig-zags.



Normalized: the contours are round, the gradient points to the minimum, and a larger α works.

> Normalizing never hurts. It matters most when the features have very different ranges.

Initialization of Parameters

Initialize Parameters

Initial parameters of the neural network can affect the **gradients** and learning

Idea 1: Constant initialization

- *Result*: For fully connected layers: identical gradients, identical neurons. Bad!

Idea 2: Random weights, to break symmetry

- Too **large** of initialization: **exploding** gradients
- Too **small** of initialization: **vanishing** gradients

Vanishing and exploding gradients

A very deep network with $g(z) = z$ and $\mathbf{b}^{[l]} = \mathbf{0}$ computes a product of matrices:

$$\hat{y} = \mathbf{X}\,\mathbf{W}^{[1]}\mathbf{W}^{[2]}\dots\mathbf{W}^{[L]}$$

Weights a little larger than 1

$$\mathbf{W}^{[l]} = \begin{pmatrix} 1.5 & 0 \\ 0 & 1.5 \end{pmatrix} \Rightarrow \hat{y} \propto 1.5^{L}$$

$L = 150$: $1.5^{150} \approx 10^{26}$. Activations and gradients **explode**.

Weights a little smaller than 1

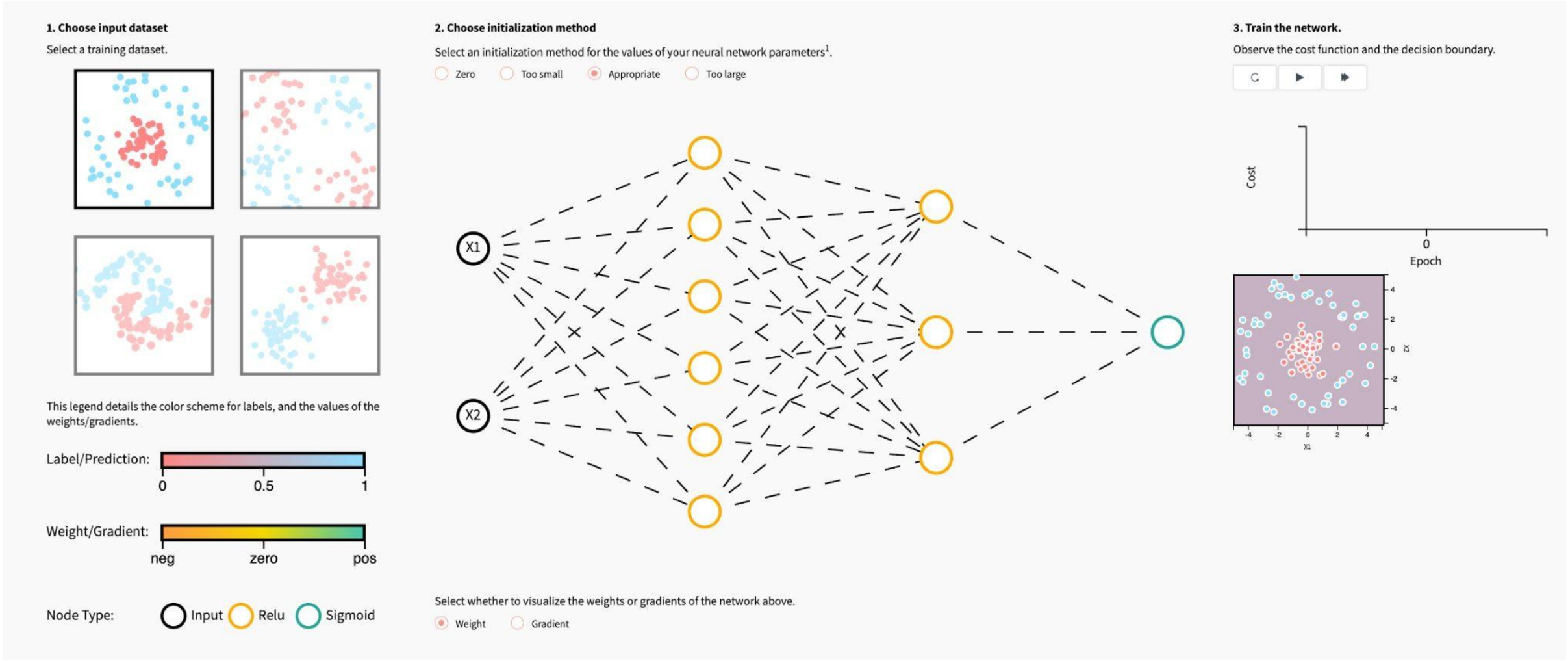
$$\mathbf{W}^{[l]} = \begin{pmatrix} 0.5 & 0 \\ 0 & 0.5 \end{pmatrix} \Rightarrow \hat{y} \propto 0.5^{L}$$

$L = 150$: $0.5^{150} \approx 10^{-45}$. Activations and gradients **vanish**.

> Gradients behave the same way, so learning becomes unstable or extremely slow. A partial solution: choose the scale of the initial weights carefully.

Interactive Demo: Initialization

Initialize Parameters



Source: Initializing neural networks. https://www.deeplearning.ai/ai-notes/initialization/

Weight initialization for deep networks

One neuron: $z = w_1x_1 + w_2x_2 + \cdots + w_nx_n$. The more inputs n, the smaller each w_i should be, so that z keeps a reasonable size: $\mathrm{Var}(w_i) \approx 1/n$.

Activation	Scale of $\mathbf{W}^{[l]}$	Name
ReLU	$\sqrt{2/n^{[l-1]}}$	He
tanh	$\sqrt{1/n^{[l-1]}}$	Xavier
tanh	$\sqrt{2/(n^{[l-1]} + n^{[l]})}$	Glorot

```
W = torch.randn(n_prev, n_l) * (2/n_prev)**0.5
nn.init.kaiming_normal_(w, nonlinearity="relu")
```

Two popular initialization schemes:

1: **Xavier** Init: For **Tanh** activation

$$W^{[l]} \sim \mathcal{N}(\mu = 0, \sigma^2 = \frac{1}{n^{[l-1]}}) \quad \text{or} \quad W^{[l]} \sim \mathcal{N}(0, \frac{2}{n^{[l-1]} + n^{[l]}})$$

$$b^{[l]} = 0$$

2. **Kaiming He** Init: **ReLU** activation

$$W^{[l]} \sim \mathcal{N}(\mu = 0, \sigma^2 = \frac{2}{n^{[l-1]}})$$

$$b^{[l]} = 0$$

of neurons in layer $l-1$

Numerical approximation of gradients

Example: $f(\theta) = \theta^3$ at $\theta = 1$, with $\varepsilon = 0.01$. The true derivative is $g(\theta) = 3\theta^2 = 3$.

One-sided difference

$$\frac{f(\theta+\varepsilon) - f(\theta)}{\varepsilon} = \frac{1.01^3 - 1}{0.01} = 3.0301$$

Error 0.0301, of order $O(\varepsilon)$.

Two-sided difference

$$\frac{f(\theta+\varepsilon) - f(\theta-\varepsilon)}{2\varepsilon} = \frac{1.01^3 - 0.99^3}{0.02} = 3.0001$$

Error 0.0001, of order $O(\varepsilon^2)$.

$$f'(\theta) = \lim_{\varepsilon \to 0} \frac{f(\theta+\varepsilon) - f(\theta-\varepsilon)}{2\varepsilon}$$

> Use the **two-sided** difference: twice the cost, much more accurate. It is how we check that a backward pass is correct.

Gradient checking

1. Reshape $\mathbf{W}^{[1]}, \mathbf{b}^{[1]}, \ldots, \mathbf{W}^{[L]}, \mathbf{b}^{[L]}$ into one big vector θ, so the cost is $J(\theta)$.
2. Reshape $d\mathbf{W}^{[1]}, d\mathbf{b}^{[1]}, \ldots$ the same way into $d\theta$. Question: is $d\theta$ the gradient of $J(\theta)$?

$$d\theta_{\text{approx}}[i] = \frac{J(\theta_1, \ldots, \theta_i + \varepsilon, \ldots) - J(\theta_1, \ldots, \theta_i - \varepsilon, \ldots)}{2\varepsilon} \approx d\theta[i] = \frac{\partial J}{\partial \theta_i}$$

$$\frac{\|d\theta_{\text{approx}} - d\theta\|_2}{\|d\theta_{\text{approx}}\|_2 + \|d\theta\|_2}$$

Relative difference, $\varepsilon = 10^{-7}$	**Verdict**
$\approx 10^{-7}$	great
$\approx 10^{-5}$	look carefully at the components
$\approx 10^{-3}$	worry: probably a bug

Gradient checking: implementation notes

- **Only to debug**, never during training: it needs two forward passes per parameter.
- If the check fails, look at the components: which $d\mathbf{W}^{[l]}$ or $d\mathbf{b}^{[l]}$ differs? That locates the bug.
- **Remember regularization:** J includes $\frac{\lambda}{2m}\sum_l \|\mathbf{W}^{[l]}\|_F^2$, so $d\theta$ must include its gradient.
- **Does not work with dropout** (J is random): check with keep_prob = 1, then turn dropout on.
- Run it at random initialization, and perhaps again after some training (some bugs only show when $\mathbf{W}, \mathbf{b}$ are far from 0).

Summary

- Dev and test sets from the same distribution. **Bias**: training error vs. optimal error. **Variance**: dev error vs. training error. Fix bias first.
- Prefer a big network with regularization to a small network.
- Regularization: L2 (weight decay, $d\mathbf{W}^{[l]} \mathrel{+}= \frac{\lambda}{m}\mathbf{W}^{[l]}$), inverted dropout (off at test time), data augmentation, early stopping.
- Setup: normalize inputs with training statistics; initialize with $\mathrm{Var}(w) = 2/n$ (He, ReLU) or $1/n$ (Xavier, tanh); check gradients.

Next week, Lecture 4: optimization algorithms: mini-batches, momentum, RMSprop, Adam, learning rate decay.



Thank you!